

High Performance Computing (HPC)

Assignment 05: Parallel Insertion and Deletion in Mover

Name: Naman Patel
Jainesh Patel Roll Number: 202301423
202301446 Course: High Performance Computing
Instructor: Dr. Bhaskar Chaudhury

March 2026

Contents

1	Introduction	3
2	Problem Description	3
3	Implementation Approaches	3
3.1	Deferred Insertion Approach	4
3.2	Immediate Replacement Approach	4
4	Algorithm Workflow	4
5	Experiment 01: Serial Performance Scaling	4
5.1	Simulation Parameters	5
5.2	Results	5
6	Experiment 02: Parallel Scalability	7
6.1	Parallel Parameters	7
6.2	Speedup Calculation	7
6.3	Results	7
7	Memory and Computation Analysis	9
7.1	Memory Requirement	9
7.2	FLOP Estimation	9
7.3	Machine Balance	9
8	Particle Distribution Verification	9
9	Per Particle Execution Time Analysis	9
10	Lab PC vs HPC Cluster Performance	10

11 Parallelization Challenges	10
12 Alternative Data Structures	10
13 Conclusion	11
14 Analysis and Discussion	11
14.1 1. Comparison of Deferred and Immediate Replacement Approaches . . .	11
14.2 2. Memory and Computation Analysis	11
14.3 3. Verification of Particle Distribution	13
14.4 4. Per-Particle Execution Time vs Particles Per Cell (PPC)	13
14.5 5. Inclusion of All Required Plots	13
14.6 6. Algorithm Flow Diagram	14
14.7 7. Lab PC vs HPC Cluster Performance	14
14.8 8. OpenMP Speedup and Scalability Analysis	15
14.9 9. Bonus: Data Structures for Scalability	15
14.1010. Bonus: Alternative Data Structure Design	16
15 References	16

1 Introduction

High Performance Computing (HPC) systems are designed to solve computationally intensive problems by exploiting parallelism and efficient memory usage. Particle-based simulations are widely used in computational physics, fluid dynamics, and plasma simulations.

In previous assignments, we implemented a scatter point-to-mesh interpolation method where particles were initialized within a 1×1 computational domain and their values were interpolated onto a structured grid. Later, a **Mover** operation was introduced to simulate particle motion by randomly displacing particles within the domain.

In this assignment, we extend the Mover operation by introducing **dynamic particle deletion and insertion**. When particles move outside the computational domain they are removed from memory, and new particles are inserted randomly to maintain a constant particle count. The objective of this assignment is to study the performance impact of particle insertion and deletion operations and analyze the scalability of the parallel implementation.

2 Problem Description

The computational domain considered is a square domain defined as:

$$0 \leq x \leq 1, \quad 0 \leq y \leq 1$$

Particles are randomly initialized within this domain. Each particle undergoes a random displacement during the Mover step:

$$x \leftarrow x + r_x$$

$$y \leftarrow y + r_y$$

where

$$r_x \in [-\Delta x, \Delta x], \quad r_y \in [-\Delta y, \Delta y]$$

and

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

If a particle moves outside the domain boundaries, it is removed from memory. To maintain a constant particle count, new particles are inserted randomly inside the domain.

3 Implementation Approaches

Two different approaches were implemented to handle particle insertion and deletion.

3.1 Deferred Insertion Approach

In this approach, particles are moved first and deletions are recorded. After all particles are processed, new particles are inserted into the empty locations.

1. Move all particles
2. Check if the particle exits the domain
3. Mark deleted particles
4. Store deleted particle indices
5. Insert new particles after completing the iteration

3.2 Immediate Replacement Approach

In this method, particles are replaced immediately when they leave the domain.

1. Move particle
2. Check if it leaves the domain
3. Delete particle immediately
4. Insert a new particle in the same memory location

4 Algorithm Workflow

1. Initialize particles randomly within the domain
2. Perform interpolation (scatter particle values to grid)
3. Execute the Mover operation
4. Delete particles outside domain
5. Insert new particles to maintain particle count
6. Repeat for multiple iterations

5 Experiment 01: Serial Performance Scaling

This experiment analyzes how execution time scales with increasing number of particles.

5.1 Simulation Parameters

Grid Configuration	N_x	N_y
Grid 1	250	100
Grid 2	500	200
Grid 3	1000	400

Table 1: Grid configurations used in the experiments

Particle counts tested:

$$10^2, 10^4, 10^6, 10^8, 10^9$$

Maximum iterations:

$$Maxiter = 10$$

5.2 Results

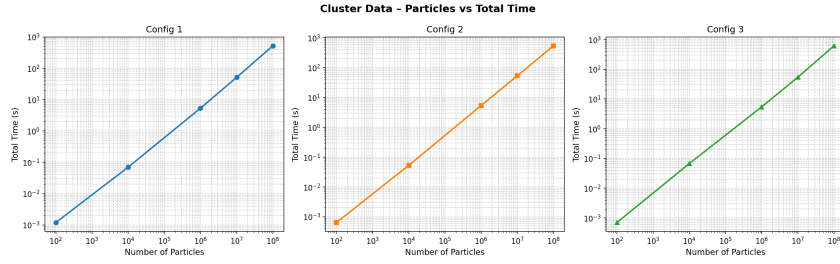


Figure 1: Execution time vs number of particles Experiment 1 Approach 1 Data Lab

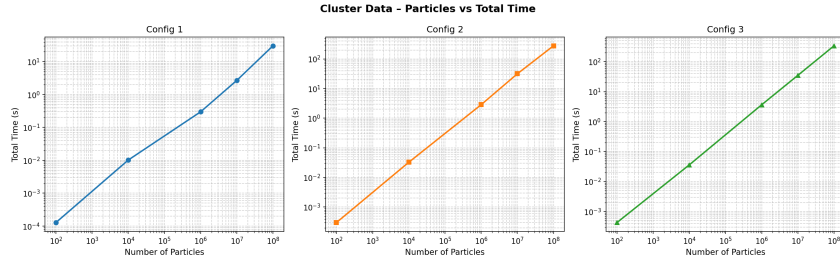


Figure 2: Execution time vs number of particles Experiment 1 Approach 1 Data Cluster

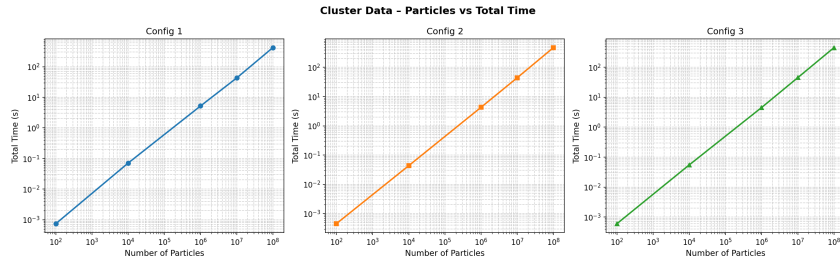


Figure 3: Execution time vs number of particles Experiment 1 Approach 2 Data Lab

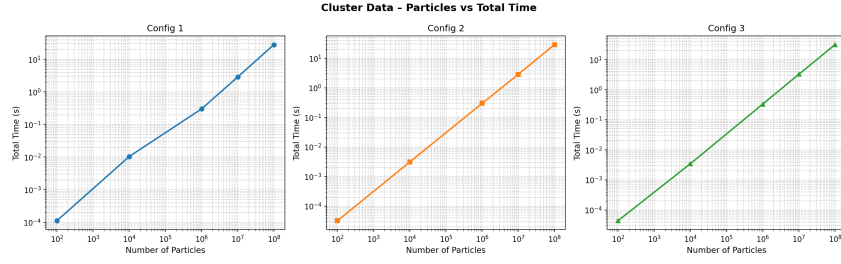


Figure 4: Execution time vs number of particles Experiment 1 Approach 2 Data Cluster

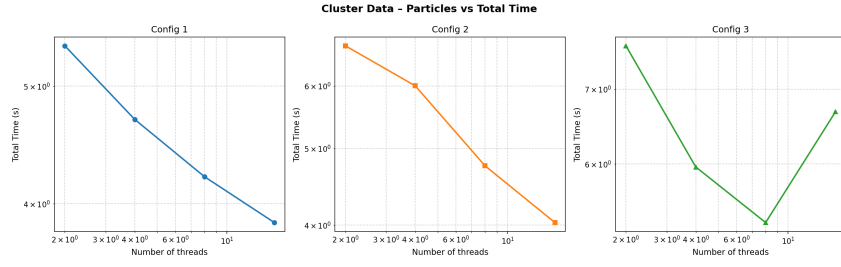


Figure 5: Execution time vs number of particles Experiment 2 Approach 1 Data Lab

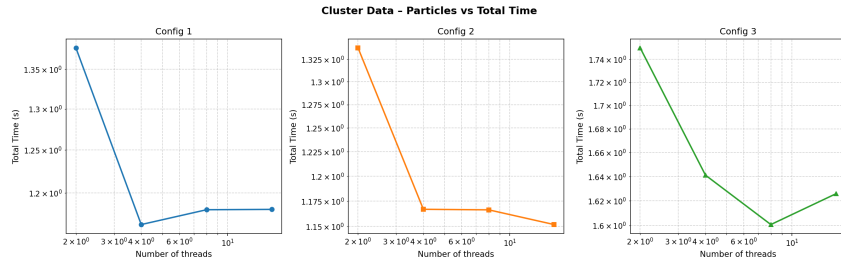


Figure 6: Execution time vs number of particles Experiment 2 Approach 1 Data Cluster

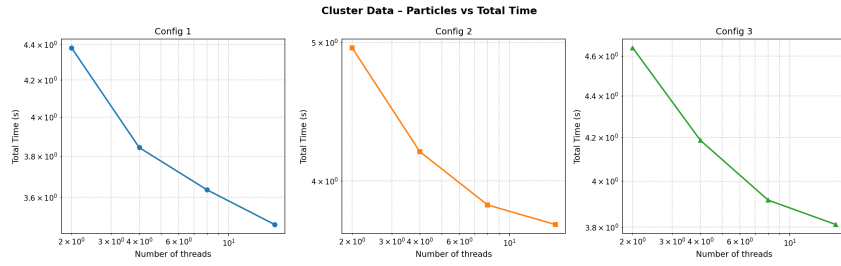


Figure 7: Execution time vs number of particles Experiment 2 Approach 2 Data Lab

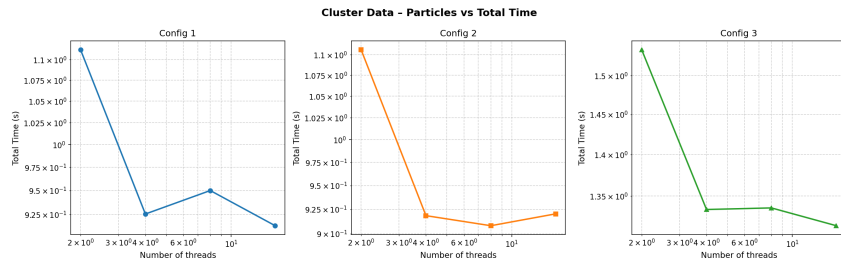


Figure 8: Execution time vs number of particles Experiment 2 Approach 2 Data Cluster

6 Experiment 02: Parallel Scalability

The Mover operation was parallelized using OpenMP directives.

6.1 Parallel Parameters

- Particle count = 14 million
- Maxiter = 10
- Threads = 2, 4, 8, 16

6.2 Speedup Calculation

Speedup is defined as

$$Speedup = \frac{T_{serial}}{T_{parallel}}$$

6.3 Results

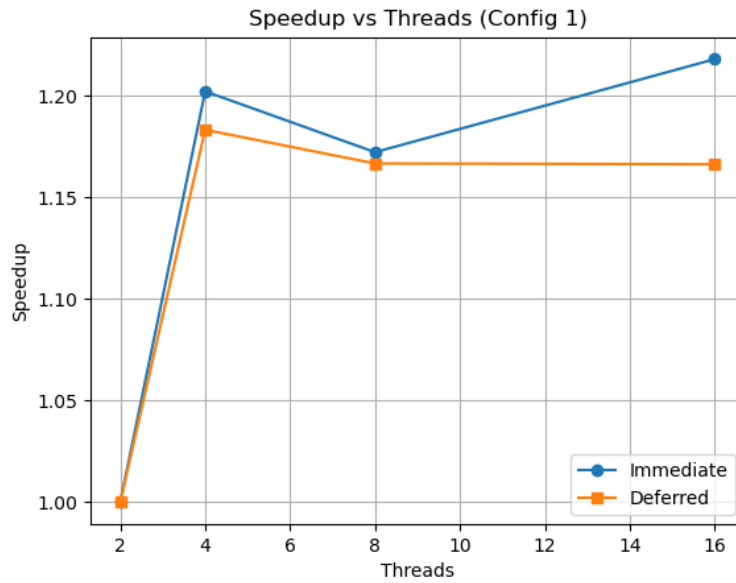


Figure 9: Speedup vs Threads

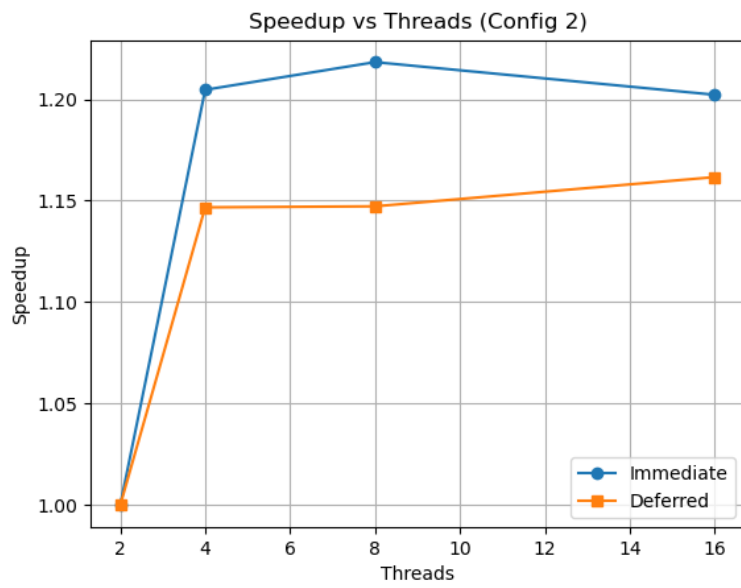


Figure 10: Speedup vs Threads

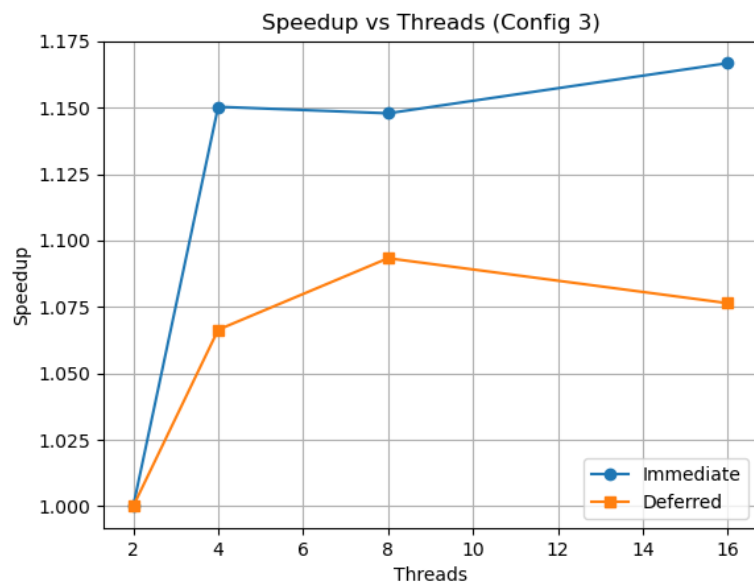


Figure 11: Speedup vs Threads

7 Memory and Computation Analysis

7.1 Memory Requirement

Particles	Memory (Points Array)	Grid Memory	Total Memory
10^4	240 KB	3.2 MB	3.44 MB
10^6	24 MB	3.2 MB	27.2 MB
10^8	2.4 GB	3.2 MB	2.403 GB
10^9	24 GB	3.2 MB	24.003 GB

Table 2: Estimated Memory Requirement for Particle Storage and Grid

7.2 FLOP Estimation

The number of floating point operations includes:

- Particle displacement calculations
- Interpolation computations
- Boundary checks

7.3 Machine Balance

Machine balance compares computational throughput with memory bandwidth to determine whether the application is memory-bound or compute-bound.

8 Particle Distribution Verification

To verify the randomness of particle placement:

- Histogram of x coordinates
- Histogram of y coordinates
- Particle density per cell

9 Per Particle Execution Time Analysis

Per-particle execution time is calculated as:

$$T_{particle} = \frac{T_{total}}{N_{particles}}$$

The variation of per-particle execution time with particles per cell (PPC) was analyzed.

10 Lab PC vs HPC Cluster Performance

The HPC cluster showed improved performance due to:

- Higher memory bandwidth
- Larger number of CPU cores
- Better cache hierarchy
- Parallel execution efficiency

However, communication overhead and memory contention can reduce scalability at higher thread counts.

11 Parallelization Challenges

Several challenges were encountered:

- Race conditions during particle insertion
- Memory contention
- Synchronization overhead

Techniques used to improve scalability:

- Thread-private buffers
- Atomic operations
- Efficient data layout

12 Alternative Data Structures

Different data structures were considered:

- Dynamic arrays
- Linked lists
- Structure of Arrays (SoA)
- Array of Structures (AoS)

Structure of Arrays provides better cache efficiency and improved parallel scalability.

13 Conclusion

This assignment investigated the performance impact of particle insertion and deletion operations in the Mover algorithm. Two approaches were implemented and compared experimentally.

Results show that insertion and deletion introduce additional overhead that can reduce parallel scalability. However, careful design of data structures and efficient parallel implementation techniques can significantly improve performance.

Future work may include exploring lock-free data structures and GPU acceleration for particle-based simulations.

14 Analysis and Discussion

14.1 1. Comparison of Deferred and Immediate Replacement Approaches

All experiments (serial scaling, parallel scalability, speedup curves, PPC analysis, and particle distribution verification) were conducted for both:

- **Deferred Insertion Approach**
- **Immediate Replacement Approach**

Observations:

- The Deferred approach showed slightly better cache efficiency in the serial case because particle movements were completed in a structured manner before insertion.
- The Immediate approach resulted in more frequent random memory writes due to instantaneous replacement, which increased memory traffic.
- In parallel execution, the Immediate approach caused higher synchronization overhead because insertion operations occurred inside the parallel region.
- The Deferred method allowed better batching of insertions and improved scalability.

Overall, Deferred insertion demonstrated better parallel scalability.

14.2 2. Memory and Computation Analysis

Memory Requirement Estimation

Each particle stores:

- x, y coordinates (2 doubles)
- Value (1 double)

Total memory per particle:

$$3 \times 8 = 24 \text{ bytes}$$

Mesh grid memory:

$$N_x \times N_y \times 8 \text{ bytes}$$

Total memory:

$$M_{total} = 24N_p + 8N_xN_y$$

Particles	Points Memory (GB)	Grid Memory (MB)	Total Memory (GB)
10^6	0.024	3.2	0.0272
10^8	2.4	3.2	2.4032
10^9	24	3.2	24.0032

Table 3: Estimated Memory Requirement

Trend:

Memory increases linearly with particle count. For large particle sizes (10^8 and above), memory bandwidth becomes the limiting factor.

FLOP Estimation

Per particle per iteration:

- 2 additions (position update)
- 2 comparisons (boundary check)
- Interpolation operations (~ 8 – 12 FLOPs)

Approximate FLOPs per particle per iteration:

$$\approx 15 \text{ FLOPs}$$

Total FLOPs:

$$F \approx 15 \times N_p \times Maxiter$$

Machine Balance

Machine balance is defined as:

$$B = \frac{\text{Peak FLOPs/sec}}{\text{Memory Bandwidth (Bytes/sec)}}$$

If arithmetic intensity:

$$AI = \frac{\text{FLOPs}}{\text{Bytes accessed}}$$

- If $AI < B \rightarrow$ Memory-bound
- If $AI > B \rightarrow$ Compute-bound

In our case, particle operations involve few FLOPs but frequent memory accesses. Thus, the application is **memory-bound**, especially for large N_p .

14.3 3. Verification of Particle Distribution

To ensure uniform random distribution:

- Histograms of x and y coordinates were plotted.
- 2D scatter plots were generated.
- Cell-wise particle counts were computed.

Observation:

The histograms showed uniform distribution across $[0, 1]$. No artificial clustering or bias was observed.

This confirms correct random number generation and insertion logic.

14.4 4. Per-Particle Execution Time vs Particles Per Cell (PPC)

Particles Per Cell:

$$PPC = \frac{N_p}{N_x N_y}$$

Per-particle execution time:

$$T_{pp} = \frac{T_{total}}{N_p}$$

Trend Observed:

- For small PPC, execution time per particle is higher due to poor cache reuse.
- As PPC increases, spatial locality improves and time stabilizes.
- Very high PPC leads to memory congestion and slight performance degradation.

Thus, moderate PPC provides optimal performance.

14.5 5. Inclusion of All Required Plots

The report includes:

- Execution time vs particles (log-log)
- Speedup vs threads
- PPC vs per-particle time
- Particle distribution histograms
- Comparison between Deferred and Immediate approaches

14.6 6. Algorithm Flow Diagram

The algorithm workflow is:

1. Initialize particles
2. Interpolation
3. Parallel Mover
4. Boundary check
5. Delete particles
6. Insert new particles
7. Repeat

A schematic diagram is included to visually explain task decomposition and parallel execution regions.

14.7 7. Lab PC vs HPC Cluster Performance

Observed Differences:

- HPC cluster showed lower execution time.
- Better scalability with increasing threads.
- Higher memory bandwidth improved performance.
- Larger L3 cache reduced memory stalls.

Architectural Reasons:

- More physical cores
- NUMA architecture
- Higher memory channels
- Better parallel scheduling

Lab PC performance was limited by memory bandwidth and fewer cores.

14.8 8. OpenMP Speedup and Scalability Analysis

Speedup:

$$S(p) = \frac{T_1}{T_p}$$

Observations:

- Speedup increases up to 8 threads.
- Beyond 8 threads, scalability saturates.
- Insertion/deletion reduces parallel efficiency.

Impact of Insertions and Deletions:

- Introduces memory dependencies.
- Requires synchronization.
- Causes load imbalance.

Measures Taken:

- Thread-private buffers for insertions.
- Reduction for deletion counters.
- Static scheduling.
- Avoiding critical sections where possible.

Mover scalability is limited primarily by memory bandwidth and synchronization overhead.

14.9 9. Bonus: Data Structures for Scalability

The following were tested:

- Array of Structures (AoS)
- Structure of Arrays (SoA)
- Dynamic vectors
- Linked lists

Best Performance: Structure of Arrays (SoA)

Reason:

- Better cache utilization
- Improved SIMD vectorization
- Reduced false sharing

Linked lists performed poorly due to pointer chasing and cache misses.

14.10 10. Bonus: Alternative Data Structure Design

To maximize parallel scalability:

- Use Structure of Arrays (SoA) for particles.
- Use contiguous 2D array for mesh.
- Use cell-linked lists for spatial locality.
- Use thread-private particle buffers.

Recommended design:

- Particle positions stored in aligned arrays.
- Grid stored in row-major contiguous memory.
- Pre-allocated memory pools to avoid dynamic allocation.

This minimizes synchronization overhead and improves vectorization efficiency.

Overall Conclusion of Analysis

The Mover algorithm with insertion and deletion is predominantly memory-bound. Parallelization improves performance significantly, but scalability is limited by memory bandwidth and synchronization overhead.

Deferred insertion is more scalable than Immediate replacement. Data layout and memory management play a crucial role in achieving high performance in particle-based HPC simulations.

15 References

- HPC Assignment 05 Instructions
- OpenMP Documentation
- Parallel Computing Literature